

Day 1: Setting Up the Kitchen & Pantry (CodeVault Backend)

Welcome to Day 1 of building CodeVault! Think of building an app like running a restaurant.

- **Spring Boot is your Kitchen.** It's where all the magic happens, where the chefs (your code) take orders and cook up responses.
- **The Database (MySQL) is your Pantry.** It's where you store all your ingredients (your data, like Code Snippets) safely so the Kitchen can grab them whenever needed.

Let's go through exactly what we did to set this up from scratch.

1. Initializing the Kitchen (Spring Boot Project)

To build our Kitchen, we use a magical blueprint generator called **Spring Initializr**. Here is what we picked:

- **Project Type (Maven):** This is our Kitchen Manager. If our kitchen needs a new blender (a library), Maven goes out, buys it, and puts it on the counter for us.
 - **Language (Java):** This is the language our chefs speak.
 - **Dependencies (The tools we added to our kitchen):**
 1. **Spring Web:** This is the "Order Window." It allows our kitchen to talk to the outside world.
 2. **Spring Data JPA:** This is our "Pantry Manager." It's a robot that knows how to go into the Pantry, find the exact shelf, and grab the data we need so we don't have to look for it manually.
 3. **MySQL Driver:** This is the special "Key" that unlocks our specific brand of Pantry (a MySQL database).
-

2. Connecting the Kitchen to the Pantry

Right now, our Kitchen is built, but it doesn't know where the Pantry is. We configure this in a file called `application.properties`.

File: `src/main/resources/application.properties`

```
# This is the map to the Pantry. localhost:3306 means the Pantry is in our own house.
# /codevault is the specific room in the pantry.
spring.datasource.url=jdbc:mysql://localhost:3306/codevault

# This is the username to unlock the Pantry door.
spring.datasource.username=root

# This is the password for the Pantry door.
spring.datasource.password=root

# This tells our Pantry Manager: "Whenever we turn on the Kitchen, check the Pantry.
# If we need a new shelf, automatically build it for us (update the pantry)."
spring.jpa.hibernate.ddl-auto=update

# This tells our Pantry Manager to talk out loud. It will print out exactly what it
```

```
is doing.  
spring.jpa.show-sql=true
```

3. Creating the Blueprint for our Ingredients

In our Pantry, we can't just throw things in a pile. We need specific boxes. A Java **Entity** is a blueprint that tells the Pantry exactly what shape the box should be. We want to store Code Snippets, so we made a `Snippet` box.

File: `src/main/java/com/example/codevault_backend/Snippet.java`

```
package com.example.codevault_backend;  
  
// We are importing tools that let us talk to the database  
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
  
// @Entity tells the Kitchen: "Hey, this is a blueprint for a table in our database  
Pantry!"  
@Entity  
public class Snippet {  
  
    // @Id means this is the unique barcode for every snippet. No two snippets have  
    the same ID.  
    @Id  
    // @GeneratedValue tells the database to automatically count up the IDs for us  
    (1, 2, 3...)   
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    // A space to store the title of the code snippet (like "How to sort an array")  
    private String title;  
  
    // A space to store the actual code itself  
    private String codeContent;  
  
    // A space to store what language the code is in (like "Java" or "Python")  
    private String language;  
  
    // This is an empty "constructor". It is like a blank order ticket the kitchen  
    needs to start working.  
    public Snippet() {  
    }  
  
    // Below are the "Getters" and "Setters".  
    // They are little windows that let the rest of our app look at or change the  
    data inside this file.  
  
    public Long getId() {
```

```

        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getCodeContent() {
        return codeContent;
    }

    public void setCodeContent(String codeContent) {
        this.codeContent = codeContent;
    }

    public String getLanguage() {
        return language;
    }

    public void setLanguage(String language) {
        this.language = language;
    }
}

```

Summary

We have successfully built our Kitchen (Spring Boot), got the key to the Pantry (MySQL application.properties), and created a blueprint (Snippet.java) so the Kitchen knows exactly how to store ingredients!